$\mathcal{L}_1$-Adaptive Augmentation for Robust Morphing-Wing Drone Flight Control

Carletti Simone
Author

Wanner Julius, Jeger Simon, Averta Giuseppe, Floreano Dario
Supervisors

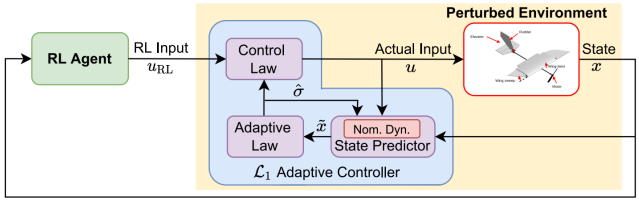

Fig. 1: Hierarchical RL + $\mathcal{L}_1$ adaptive control architecture.

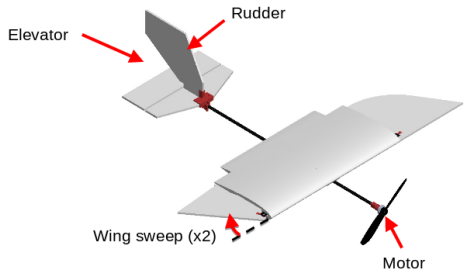

Fig. 2: The custom morphing-wing drone (Lisparrow).

I. Introduction

The field of Unmanned Aerial Vehicle (UAV) control is shifting toward learning-based methods, particularly **Reinforcement Learning (RL)**. RL enables training of agile control policies in simulation, but the **sim-to-real gap** caused by unmodeled dynamics, parameter mismatches, and disturbances remains a major challenge. Techniques like **Domain Randomization (DR)** improve robustness by training across varied parameters, but they can produce overly conservative policies, often fail when the training model is inaccurate, and fundamentally cannot cover all possible real-world disturbances and unmodeled dynamics. This motivates the need for low-level adaptive controllers that compensate for uncertainties in real time. This thesis proposes a hybrid approach combining **$\mathcal{L}_1$ Adaptive Control** with a **high-level RL policy** to retain learning-based agility while ensuring robustness to model uncertainties and external disturbances.

II. Related Work

While **Reinforcement Learning (RL)** provides a lightweight alternative to computationally heavy traditional methods, mitigating the *Sim-to-Real gap* remains a critical challenge.

To handle real-time uncertainties without sacrificing the agility of learning-based methods, researchers have investigated hybrid architectures that incorporate **Adaptive Control**. Traditional Model Reference Adaptive Control (MRAC) has been utilized to ensure trajectory tracking, though it traditionally suffers from a trade-off between adaptation speed and system robustness. $\mathcal{L}_1$ Adaptive Control ($\mathcal{L}_1$-AC) addresses this by decoupling adaptation from control, permitting fast adaptation rates without exciting high-frequency unmodeled dynamics. Hanover et al. successfully **demonstrated the efficacy** of $\mathcal{L}_1$-AC on quadrotors by combining it with NMPC, significantly improving precision and payload handling[1].

More recently, integrating $\mathcal{L}_1$-AC directly with RL policies has shown promise on simpler systems[2]. Building upon this foundation, this work extends the framework to winged UAVs by proposing a **hierarchical approach coupling a high-level RL policy with low-level $\mathcal{L}_1$-AC** to guarantee both agility and reliability in flight (see fig. 1).

[1]Hanover et al., "Performance, precision, and payloads: Adaptive nonlinear mpc for quadrotors." IEEE RA-L - 2021

[2]Cheng et al., "Improving the robustness of reinforcement learning policies with L1 adaptive control," IEEE RA-L - 2022

III. Methodology

A. System Overview and Modeling

The platform utilized in this work is a **custom avian-inspired morphing drone** (see fig. 2) developed by the Laboratory of Intelligent Systems (LIS) of EPFL. Unlike standard rigid-frame drones, this platform features **morphing wings** which allows it to alter its aerodynamic profile in flight.

The vehicle is modeled as a 6-DOF rigid body, whose dynamics are described by the Newton–Euler equations. For control purposes, we use the reduced state

$$\mathbf{x} = \left[\mathbf{v}^\top, \boldsymbol{\omega}^\top\right]^\top \tag{1}$$

where $\mathbf{v}$ and $\boldsymbol{\omega}$ denote the linear and angular velocities in the body frame. The control vector is

$$\mathbf{u} = [T, \mathrm{sw}_L, \mathrm{sw}_R, \mathrm{ele}, \mathrm{rud}]^\top, \tag{2}$$

representing thrust (in Newtons), left wing sweep, right wing sweep, elevator, and rudder deflections (in degrees), all subject to strict actuator limits. Notably, the entire set of actuators features **limited operational ranges and non-negligible dynamics**, making the platform significantly more constrained than a standard quadrotor.

B. High-Level Controller: Reinforcement Learning

The high-level controller is a Reinforcement Learning (RL) policy trained using Proximal Policy Optimization (PPO). Importantly, the RL policy is trained *without* the $\mathcal{L}_1$ Adaptive Controller in the loop; the low-level augmentation is only applied during deployment to improve Sim-to-Real transfer.

The observation vector is defined as:

$$o_t = \left[\mathbf{p}_{g,t}^\top, \mathrm{vec}(\mathbf{R}_t)^\top, \mathbf{v}_t^\top, \boldsymbol{\omega}_t^\top\right]^\top, \tag{3}$$

where $\mathbf{p}_{g,t}$ is the relative position to the next trajectory waypoint, $\mathbf{R}_t$ is the body-frame rotation (flattened), and $\mathbf{v}_t$, $\boldsymbol{\omega}_t$ are the linear and angular velocities. The RL agent outputs a normalized action vector $a_t \in [-1, 1]^5$, mapped to physical actuators using

$$u_t = \mu + \sigma\, a_t, \qquad \mu = \frac{u_{\max} + u_{\min}}{2}, \quad \sigma = \frac{u_{\max} - u_{\min}}{2}. \tag{4}$$

The reward function guiding the RL agent is defined as

$$r_t = r_{\text{traj}} + r_{\text{stab}} + r_{\text{eff}} + r_{\text{term}}, \tag{5}$$

encouraging accurate trajectory tracking, stability, efficiency, and penalizing terminal states.

C. Low-Level Controller: $\mathcal{L}_1$ Adaptive Control

To ensure robustness against unmodeled dynamics and disturbances, the high-level RL commands are augmented with a low-level $\mathcal{L}_1$ Adaptive Controller. This controller actively compensates for uncertainties and guarantees stable tracking. Its main components are described below.

1. State Predictor

For a discrete-time implementation, the state predictor **estimates the system's evolution** under nominal dynamics as:

$$\hat{x}_{k+1} = \hat{x}_k + (f(x_k, u_k) + \hat{\sigma}_k + A_s \tilde{x}_k)\,\Delta t \tag{6}$$

where $\tilde{x}_k \triangleq \hat{x}_k - x_k$ is the prediction error, $\hat{\sigma}_k$ represents generalized uncertainties, Δt is the integration step, and A_s is a non-positive diagonal matrix that defines the adaptation and predictor dynamics.

2. Adaptation Law

The adaptation law **estimates the lumped disturbances** $\hat{\sigma}_k$ based on the prediction error:

$$\hat{\sigma}_k = \Gamma \tilde{x}_k, \quad \Gamma \triangleq -\left(e^{A_s \Delta t} - I\right)^{-1} A_s e^{A_s \Delta t}. \tag{7}$$

Since we are operating in discrete time, we can define the adaptation law as a simple piecewise-constant gain Γ.

2.1. Control Allocation via QP

The **estimated disturbances are then mapped to control space** in order to obtain a corrective signal $\hat{\sigma}_m$. While traditionally this is done via a simple pseudoinverse, our system's strict actuator limits and inherent delays require a more robust approach. Therefore, the allocation is solved through a constrained Quadratic Programming (QP) optimization problem:

$$\min_{\hat{\sigma}_m} \ \frac{1}{2}\|B\hat{\sigma}_m - \hat{\sigma}_k\|_2^2 + \mathcal{R}(\hat{\sigma}_m) \tag{8}$$

$$\text{s.t. } u_{\min} - u_{RL} \le \hat{\sigma}_m \le u_{\max} - u_{RL}, \tag{9}$$

where B is the sensitivity matrix, computed online using central difference approximation. The regularization term $\mathcal{R}(\hat{\sigma}_m)$ penalizes excessive or rapid control efforts to ensure actuator commands remain efficient, feasible, and safe.

4. Low-Pass Filtering

The **defining feature** of $\mathcal{L}_1$-AC is the use of a low-pass filter to **decouple the adaptation speed** from the control loop's robustness. The estimated matched disturbance $\hat{\sigma}_{m,k}$ is passed through a filter to generate the adaptive control signal u_{ad}:

$$u_{AD,k} = -\text{LPF}(\hat{\sigma}_{m,k}) \quad , \quad u_k = u_{RL,k} + u_{AD,k} \tag{10}$$

where $\text{LPF}(\cdot)$ is a first-order low-pass filter with bandwidth ω_c. This ensures the controller only actively compensates for low-frequency uncertainties within the system's physical capabilities, preventing high-frequency noise from feeding into and destabilizing the actuators. Ultimately, the filtered adaptive command $u_{AD,k}$ and the high-level RL output $u_{RL,k}$ are combined to form u_k, the final control signal sent to the system.

5. Transport Delay Compensation

In our experimental setup, the controller runs off-board, which introduces transport delays. To explicitly compensate for this, the measured states are forward-propagated:

$$x_{\text{corr}}(t) = x(t) + \int_t^{t+\tau_{\text{act}}} f(x(\tau), u_{\text{hist}}(\tau))\, d\tau, \tag{11}$$

where τ_{act} is the transport delay and $u_{\text{hist}}(\tau)$ are the previously applied control inputs. This ensures that the predictor and adaptive law act on a state aligned with the real system.

IV. Experiments and Results

A. Experimental Setup

To validate the proposed control architecture, extensive evaluations were conducted in both **simulation** and **real-world** experiments. The primary objective was to assess the system's ability to maintain stable trajectory tracking under significant uncertainties, specifically mass offset, across two primary flight regimes:

- **Hover Scenario**: Maintaining a fixed setpoint in the presence of a constant 6 m/s wind.
- **Agile Scenario**: Executing a pitch-up maneuver after a 6 m/s launch.

Specifically for the **hover** scenario we compared four control configurations to isolate the contributions of DR and $\mathcal{L}_1$-AC:

- **RL (Baseline):** policy trained without DR.
- **RL + DR:** policy trained using DR.
- **RL + $\mathcal{L}_1$:** the baseline RL policy augmented with $\mathcal{L}_1$-AC.
- **RL + DR + $\mathcal{L}_1$:** the complete architecture utilizing both DR and $\mathcal{L}_1$-AC.

You can find video demonstrations for all experimental scenarios at https://go.epfl.ch/carletti-master-thesis.

B. Evaluation Metrics - Hover Scenario

The performance of each configuration was quantified using two primary metrics:

- **Mean Reward:** This metric reflects the overall task success based on the defined reward function (see (5)).
- **Root Mean Square Error (RMSE) to Goal:** This metric measures the tracking error between the drone and the desired hovering setpoint.

C. Results and Analysis - Hover Scenario

The results from the **hover scenario** demonstrate that the combination of both offline (DR) and online ($\mathcal{L}_1$) robustness techniques yields the highest performance. As shown in the comparison of Mean Total Rewards (see fig. 3), the baseline RL policy struggled significantly with +50% mass offset, while the **fully augmented** RL + DR + $\mathcal{L}_1$ **configuration achieved the best result**. Even in cases where DR is already present, the addition of $\mathcal{L}_1$ augmentation significantly **stabilizes the control signal**, reducing high-frequency oscillations and ensuring smoother actuator commands during deployment (see fig. 4). Tracking precision reflects this trend, with the combined architecture reaching the lowest RMSE to the goal compared to all other configurations (see fig. 5). Real-world validation in a *WindShape* wind tunnel confirmed these results (see fig. 6).

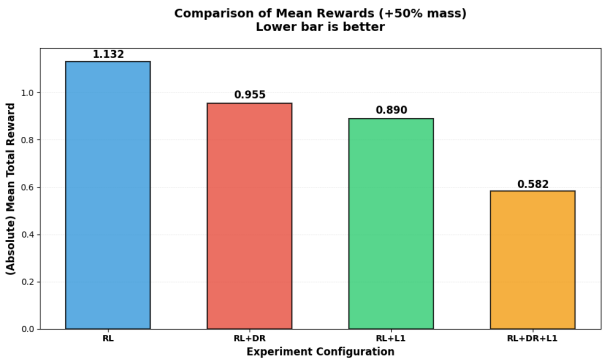

Fig. 3: Hover in Simulation - Mean Reward Comparison

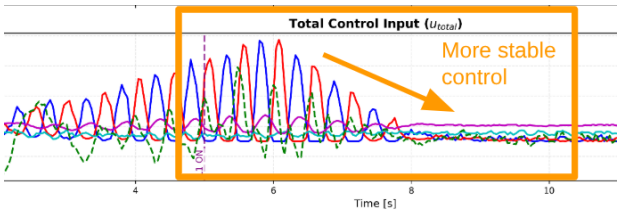

Fig. 4: Hover in Simulation (with DR) - L1 stabilizes

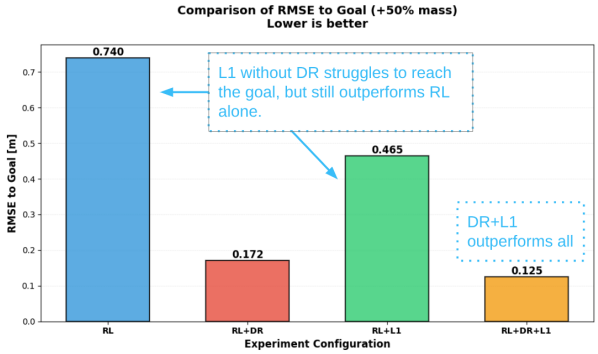

Fig. 5: Hover in Simulation - RMSE to Goal

D. Results and Analysis - Agile Scenario

To further evaluate the framework, experiments were conducted using an agile policy involving a pitch-up maneuver executed after launching the drone at 6 m/s. These tests, performed in both simulation and real-life environments, revealed a measurable correction in Z-height despite a 20% mass offset. This mass offset was specifically restricted to 20% because the drone's stringent actuator saturation limits leave limited control authority for further compensation, making the performance gains slighter than in the hover case as the system operates near its physical limits. In real-life experiments, the $\mathcal{L}_1$ controller provided **clear adjustments in Z-height and a reduction in roll error** compared to the baseline. However, the real-world $\mathcal{L}_1$ implementation showed a tendency to overcorrect. This behavior suggests that while the adaptive augmentation improves performance, it likely requires more extensive tuning to account for the specific characteristics of the physical system.

V. Discussion and Limitations

The experimental results demonstrate that the proposed hierarchical architecture, coupling Reinforcement Learning with $\mathcal{L}_1$ Adaptive Control, **effectively helps close the sim-to-real gap** by rejecting unmodeled dynamics and external disturbances. The controller successfully compensates for significant model mismatches, such as a 50% mass offset, which would otherwise destabilize a baseline RL policy.

However, the current implementation presents **notable limitations primarily tied to hardware** and system architecture. The reliance on **off-board computation** introduces a communication delay that inherently bottlenecks the real-time responsiveness of the adaptive controller. While we compensate by projecting the state forward, this **latency remains a performance constraint** due to inaccuracies in the nominal dynamics used for the projection. Furthermore, the physical constraints of the current UAV platform lead to **frequent actuator saturation**, which restricts performance: relying solely on wing sweep provides limited control authority compared to standard quadrotors; for example, if the sweep mechanism is fully saturated maintaining baseline stability or rejecting a disturbance, it cannot simultaneously execute basic navigation commands (like changing height). This fundamental actuation bottleneck restricts the full potential of the learned policy and the adaptive law's corrective capacity. Finally, the framework requires a stable high-level policy as a foundation, and while $\mathcal{L}_1$ significantly stabilizes control signals, it remains sensitive to the quality of the baseline training.

Fig. 6: Hover in real-life

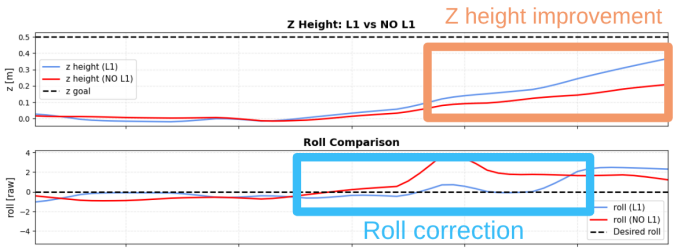

Fig. 7: Agile trajectory in real-life - L1 vs NO L1

VI. Conclusions and Future Work

The framework effectively manages real-time uncertainties and dynamic mismatches, proving the viability of this hybrid approach. While communication delays and hardware limits currently bound the maximum achievable agility, **the proposed architecture provides a robust, proven foundation for future platforms**.

Building on this framework, future research will focus on architectural and hardware improvements to further enhance system performance and reliability:

- **Onboard computation:** Transitioning to fully onboard processing will significantly reduce communication delays and improve real-time control responsiveness.
- **INDI-based control:** Integrating the current architecture with Incremental Nonlinear Dynamic Inversion is expected to yield superior disturbance rejection and increase systemic robustness in unpredictable environments, as shown by Wang et al.[3]. INDI is particularly effective at handling fast, transient disturbances such as wind gusts, complementing L1 adaptive control which excels at compensating constant model uncertainties.
- **Wing-twist prototype:** Deploying these control strategies on a new UAV prototype equipped with a wing-twist mechanism aims to directly mitigate the high actuator saturation observed during aggressive maneuvers.

[3]Wang et al., "Adaptive Augmentation of Incremental Nonlinear Dynamic Inversion for Micro Quadrotor Aerobatic Flight Control", IEEE AESS - 2026